SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1702**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

*I **S Hariram (1965260018)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

SCENARIO 1: AI-POWERED EMERGENCY MEDICAL DRONE IN A FLOOD-AFFECTED REGION

---

i. Behavior of Greedy Best-First Search (GBFS) and its Strengths & Weaknesses

Behavior: Greedy Best-First Search evaluates nodes using only the heuristic function: f(n) = h(n) where h(n) is the straight-line distance from the current waypoint 'n' to the destination. It completely ignores the actual path cost g(n) (terrain difficulty, headwinds, heavy rain, and battery usage) and always moves the drone toward the node geographically closest to the goal.

Strengths under Uncertainty:

1. Fast Computation (Low Latency): Expands very few nodes by moving directly toward the goal, enabling immediate takeoff and rapid re-computation.
2. Low Memory Overhead: Requires minimal onboard memory compared to exhaustive searches.

Weaknesses under Uncertainty:

1. Ignores Weather and Terrain Costs: Because it ignores g(n), GBFS will fly the drone directly through severe storms or high-cost wind zones if they lie on the straight-line path, risking a crash.
2. Vulnerable to Dead Ends/Blocked Paths: With partial real-time updates, the drone will fly straight toward the goal until it hits a blocked path, requiring costly physical backtracking.
3. Not Optimal: Does not guarantee the fastest, safest, or most energy-efficient route.

---

ii. How A* Search Handles the Situation (Balancing Cost and Heuristic)

Behavior & Balancing Mechanism: A* Search evaluates nodes by combining actual accumulated cost and estimated remaining cost: f(n) = g(n) + h(n)

- g(n): Actual path cost from the start node to node 'n' (accounts for variable weather, wind resistance, and terrain risk).
- h(n): Straight-line heuristic estimate from node 'n' to the goal.

How it Balances Both: If a direct straight-line route encounters severe weather or rough terrain, g(n) increases sharply. A* penalizes that hazardous path and automatically diverts the drone onto a safer detour where the combined cost f(n) = g(n) + h(n) is minimized. Since

straight-line distance never overestimates actual travel distance (admissible heuristic), A* guarantees an optimal, least-cost route for the currently known map.

---

iii. Impact of Heuristic Accuracy on Both Algorithms

1. When Heuristic Underestimates True Cost ($h(n) << h^*(n)$) due to bad weather:
    - Greedy Best-First Search: Performs poorly. Since it relies solely on h(n), an inaccurate heuristic that ignores weather misleads the drone straight into high-cost storm zones or blocked paths.
    - A* Search: Remains optimal and safe (since h(n) is still admissible), but expands more nodes and takes slightly longer to compute because it relies more heavily on g(n).
2. When Heuristic is Highly Accurate (h(n) is close to true cost $h^*(n)$):
    - Greedy Best-First Search: Finds a reasonable path quickly with minimal backtracking.
    - A* Search: Achieves peak efficiency—expanding the minimum number of nodes while still guaranteeing the optimal, safest path.
3. When Heuristic Overestimates True Cost ($h(n) > h^*(n)$, Inadmissible):
    - A* loses its strict optimality guarantee and behaves more like Greedy Search, trading slight path optimality for faster computation speed.

---

iv. How Dynamic Changes (Blocked Paths) Affect Search Performance

1. Invalidation of Current Plan: Both standard GBFS and A* assume a static graph. When real-time updates reveal a newly blocked path or sudden weather change along the planned route, the existing Open/Closed lists and path become invalid.
2. Re-planning Overhead: Standard A* must halt and recompute a new search tree from scratch starting at the drone's current location, increasing computational delay during flight.
3. Physical Backtracking Penalty: Because updates are partial, the drone may fly part of the way down a path before discovering it is blocked, wasting critical time and battery to backtrack.

---

v. Recommended Algorithm and Justification

Recommendation: A* Search (specifically Weighted A* or its dynamic variant, D* Lite / Dynamic A*).

Justification:

1. Safety: Unlike Greedy Search, A* accounts for variable terrain and weather costs via g(n), preventing the drone from flying into hazardous storm cells to save straight-line distance.
2. Optimality: Emergency drones operate on strict battery limits. A* minimizes total path cost f(n) = g(n) + h(n), ensuring optimal energy use and minimum delivery delay.
3. Real-Time Constraints: While standard A* is slower than GBFS when re-planning from scratch, using Weighted A* (f(n) = g(n) + w*h(n)) or D* Lite allows the drone to incrementally update only the blocked edges in real time without recomputing the entire graph, achieving the ideal balance of speed, optimality, and flight safety.

SCENARIO 2: SMART CITY AI TRAFFIC SIGNAL OPTIMIZATION

Problem Formulation & Why Traditional Search is Inefficient

Optimization Problem Formulation:

- State Space (S): A vector X = [t1, t2, ..., tN] representing signal timing parameters (green-light duration, cycle length, and phase offset) across all N intersections.
- Initial State: The current signal timing configuration across the city.
- Successor Function: Adjusting green/red split durations or offsets at one or more intersections by a small interval (e.g., +/- 5 seconds).
- Objective Function f(X): Minimize a weighted sum of total waiting time, fuel consumption, and congestion index: Minimize f(X) = w1*(Waiting Time) + w2*(Fuel Consumption) + w3*(Congestion)

Why Traditional Search (BFS, DFS, A*) is Inefficient:

1. Combinatorial Explosion: With hundreds of intersections (e.g., 200 intersections with 10 timing states each = 10^200 states), systematic tree search immediately exhausts memory and time limits.
2. Unknown Goal State: Traditional search requires a known target state to check against or estimate distance toward. In signal optimization, the optimal state is unknown beforehand; we only have a cost function to evaluate states.
3. Path is Irrelevant: Only the final signal timing configuration matters, not the sequence of adjustments taken to reach it.

---

i. How Hill Climbing Works in This Scenario and Its Limitations

How Hill Climbing Works:

1. Start with the current city-wide signal timing configuration and evaluate its congestion/waiting cost.
2. Generate neighboring states by slightly modifying green-light timings (e.g., +/- 5 seconds) at adjacent intersections.
3. Move greedily to the neighboring configuration that gives the largest reduction in waiting time and fuel consumption.
4. Repeat until no neighboring adjustment improves traffic flow.

Limitations:

1. Greedy and Short-Sighted: Never accepts a temporary local slowdown even if it is necessary to unlock a city-wide synchronized "green wave".
2. Trapped in Suboptimal States: Easily gets stuck in local optima, plateaus, and ridges.
3. High Sensitivity to Initial State: Cannot explore the broader state space beyond its starting neighborhood.

---

ii. Local Maxima, Plateaus, and Ridges with Real-World Interpretation

1. Local Maxima (Local Optimum):
    - Definition: A state where any single small timing adjustment worsens performance, even though a much better global configuration exists elsewhere.
    - Real-World Interpretation: A neighborhood cluster of 4 intersections is locally balanced so changing any single light by 5 seconds adds delay, preventing the system from discovering a city-wide arterial synchronization that requires shifting 30 lights at once.
2. Plateaus (Flat Regions):
    - Definition: A region of the search space where all neighboring states have the exact same objective score, offering zero gradient information.
    - Real-World Interpretation: During late-night empty roads or severe bumper-to-bumper gridlock, changing a green light from 40s to 45s produces zero change in total waiting time, leaving Hill Climbing directionless.
3. Ridges:
    - Definition: A narrow diagonal path of optimal states where changing one variable at a time drops the system into worse states on either side.

- Real-World Interpretation: Coordinating a green wave along a major highway requires updating Intersection A's duration AND Intersection B's phase offset simultaneously. Updating only one intersection creates a bottleneck at the next, causing single-step Hill Climbing to fail.

---

iii. How Simulated Annealing or Genetic Algorithms Overcome These Issues

1. Simulated Annealing (SA):
    - Uses a temperature parameter (T) to probabilistically accept worse moves with probability P = exp(-Delta_E / T).
    - At higher temperatures, SA accepts temporary local congestion increases, allowing the system to jump out of local maxima and cross ridges to find globally synchronized signal timings before cooling down.
2. Genetic Algorithms (GA):
    - Maintains a population of multiple candidate signal timing plans (chromosomes) simultaneously.
    - Uses Selection (keeping low-congestion plans), Crossover (combining optimized North-sector timings from one plan with optimized South-sector timings from another), and Mutation (randomly tweaking timings).
    - Multi-point search, crossover, and mutation allow GA to bypass local maxima, jump across plateaus, and evolve coupled intersections together along ridges.

---

iv. Recommended Approach and Justification

Recommendation: Genetic Algorithms (specifically a Distributed / Hybrid Memetic Genetic Algorithm).

Justification:

1. Scalability: Across hundreds of intersections, single-point searches (Hill Climbing/SA) struggle with the massive dimensionality. Genetic Algorithms allow regional sub-networks to be encoded as gene segments, evaluated in parallel, and combined via crossover to scale effortlessly across an entire smart city.
2. Adaptability: Because traffic patterns shift dynamically (rush hour, accidents, weather), GA's diverse population of solutions acts as a built-in memory buffer. Instead of restarting from scratch when traffic flow changes, the population rapidly adapts through mutation and crossover to converge on a new optimal timing plan in real time.

SCENARIO 3: AUTONOMOUS MARS ROVER EXPLORATION IN UNKNOWN TERRAIN

---

i. Why This Problem Requires an Online Search Agent Instead of Offline Search

- Offline Search vs. Online Search: Offline search algorithms (like standard BFS, DFS, and A*) compute a complete action sequence from start to goal in memory before taking a single physical step. Online search interleaves computation and action: the agent takes a step, observes the environment, updates its map, and computes the next move.
- Why the Mars Rover Requires Online Search:
    1. Unknown Map (Exploration Problem): The rover does not possess a complete map of craters, boulders, or impassable terrain beforehand; obstacles are discovered only when within sensor range.
    2. Empirical Step Costs: Actual traversal costs (such as wheel slip on loose sand or steep slopes) can only be determined locally as the rover observes and traverses the terrain.
    3. Communication Delay: Due to the long signal delay between Earth and Mars, the rover cannot wait for offline replanning from Earth and must make autonomous, real-time decisions on the spot.

---

ii. Challenges of Partial Observability and Dynamic Environments

1. Challenges of Partial Observability:
    - Limited Sensor Horizon & Occlusion: Rocks and hills block the rover's line of sight. A path that appears clear locally may lead to a steep crater just beyond sensor range.
    - Costly Physical Backtracking: Unlike offline search where backtracking occurs purely in memory, hitting a dead end in online search forces the rover to physically drive backward, wasting time and battery power.
    - Irreversible Dead Ends (Traps): Without full visibility, the rover risks driving down a steep crater wall or into soft sand from which it cannot climb out.
2. Challenges of Dynamic Environments:

- Changing Terrain: Dust storms, shifting sand dunes, or loose rocks can block previously clear paths.
- Fluctuating Energy Constraints: Dust on solar panels or temperature drops dynamically alter available battery power, changing which routes are safe.
- Odometry Drift: Wheel slip on Martian soil causes localization errors between expected and actual positions.

---

iii. How the Rover Builds and Updates Its Internal Model

The rover operates as a Model-Based Online Agent using SLAM (Simultaneous Localization and Mapping) and Learning Real-Time A* (LRTA*):

1. Occupancy Grid / Costmap Construction: The rover represents the terrain as a grid where each cell is labeled as Unexplored, Free, High-Cost (steep/rough), or Hazard/Obstacle (craters/rocks).
2. Sensor-Driven Graph Expansion: At each step, onboard stereo cameras and LiDAR scan the immediate surroundings, adding newly discovered states and assigning edge traversal costs c(s, a, s') based on terrain roughness and slope.
3. Dynamic Heuristic Updating (LRTA* Rule): The rover maintains a heuristic table H(s) initialized to the straight-line distance h(s) to the sample target. Whenever it moves from state 's', it updates: H(s) = min over all actions 'b' of [c(s, b, s') + H(s')] If a path leads into a blocked crater rim, H(s) increases sharply, creating a virtual elevation penalty that prevents the rover from getting stuck in loops or revisiting dead ends.

---

iv. Comparison: Online Search vs. Uninformed and Informed Search

1. Uninformed Search (Offline BFS / DFS):
    - Assumes a complete map is known upfront.
    - Blindly expands nodes without heuristic guidance.
    - Unsuitable for the rover because full terrain knowledge is unavailable and blind physical exploration wastes battery.
2. Informed Search (Offline GBFS / A*):
    - Uses heuristic h(n) to guide search efficiently toward the goal.
    - Still requires the complete state-space graph in advance to plan in memory before acting.
    - Fails if unmapped craters block the precomputed path.

3. Online Search (Online DFS / LRTA* / D* Lite):
    - Interleaves planning and physical execution step-by-step without needing a complete map upfront.
    - Discovers obstacles locally and updates its internal map and heuristic table H(s) in real time.
    - Backtracking requires physical movement, making informed online search (LRTA* / D* Lite) far superior to uninformed Online DFS.

---

v. Justification of the Most Suitable Approach

Recommendation: Learning Real-Time A* (LRTA*) or D* Lite (Dynamic A*) with a Risk-Aware Cost Function.

Justification:

1. Uncertainty: Since the Martian terrain is unknown, LRTA* and D* Lite combine optimistic heuristic guidance (straight-line distance to the sample site) for unexplored areas with real-time map updates as sensors detect hazards.
2. Adaptability: When new obstacles (craters, rocks) enter the sensor range, LRTA* and D* Lite incrementally update local state estimates H(s) without recomputing the entire global map from scratch, satisfying strict onboard real-time CPU limits.
3. Safety: By incorporating slope angle, rock density, and hazard proximity into the step cost c(s, a, s') and restricting movement strictly to reversible states within the verified sensor horizon, the rover avoids fatal traps while minimizing physical backtracking.

SCENARIO 4: UNIVERSITY EXAM TIMETABLE GENERATION (CSP)

---

i. Formulation as a Constraint Satisfaction Problem (CSP)

1. Variables (X): Each course requiring an exam is a variable: X = {C1, C2, C3, ..., Cn}.
2. Domains (D): For each course Ci, the domain Di consists of all valid pairs of Time Slots (T) and Exam Halls (H): Di = { (T1, H1), (T1, H2), ..., (Tm, Hk) }.
3. Constraints (C):
    - Hard Constraints (Must be satisfied):
        - No Student Overlap: If courses Ci and Cj share enrolled students, Ti != Tj.

    - Hall Capacity & Exclusion: No two courses can share the same hall at the same time (Hi != Hj when Ti = Tj), and course enrollment must not exceed hall capacity.
    - Precedence: Foundational subjects must be scheduled before advanced subjects (T_foundational < T_advanced).
    - Faculty Availability: Exams must align with invigilator availability.
  - Soft Constraints: Special accommodations (extra time/specific rooms) and minimizing back-to-back exams for students.

---

ii. How Backtracking Search Works in This Scenario

- Incremental Assignment: The algorithm assigns a time slot and hall to one course at a time.
- Consistency Checking: It verifies if the chosen assignment conflicts with any previously scheduled courses (e.g., shared students or overlapping hall capacity).
- Recursive Traversal: If valid, it proceeds to the next course.
- Backtracking: If a course runs out of valid time/hall combinations due to conflicts, the algorithm backtracks to the previous course, undoes its assignment, and tries the next available domain value.

---

iii. How Constraint Propagation (Forward Checking) Improves Efficiency

- Definition: When a variable Ci is assigned a value, forward checking evaluates all neighboring unassigned variables (courses sharing students) and immediately deletes conflicting time slots from their domains.
- Efficiency: It identifies dead ends early. If a neighboring course's domain becomes completely empty, the algorithm backtracks instantly without wasting time searching deeper down an invalid branch.

---

iv. Real-World Challenges and Scalability Strategy

Challenges:

- Combinatorial Explosion: Large universities with thousands of courses create massive search spaces that overwhelm naive backtracking.
- Competing Preferences: Balancing faculty schedules and student gap constraints makes optimization difficult.

Recommended Strategy for Scalability:

1. Heuristic Variable Ordering (MRV): Use the Minimum Remaining Values heuristic to pick the most constrained course first, reducing branching factor.
2. Hybrid CSP & Metaheuristics: For large-scale universities, combine CSP constraint propagation to handle hard constraints with Genetic Algorithms or Min-Conflicts local search to optimize soft constraints rapidly.

SCENARIO 5: STRATEGIC GAME AI FOR REAL-TIME DECISION MAKING (MINIMAX & ALPHA-BETA)

---

i. How the Minimax Algorithm Models the Problem

- Zero-Sum Game Tree: Models the game as a tree where the AI opponent is the Maximizer (seeking highest score) and the human player is the Minimizer (seeking lowest score).
- States and Actions: Root represents the current state; edges represent valid moves; leaf nodes represent evaluated outcomes.
- Recursive Backtracking: Utility values from terminal or heuristic leaf nodes are propagated upward (Max chooses highest child, Min chooses lowest child) to determine the optimal next move.

---

ii. Computational Challenges of Large Game Trees

1. Combinatorial Explosion (O(b^m)): As branching factor (b) and search depth (m) increase, the number of states grows exponentially, making full-depth search impossible.
2. Strict Time Constraints: Real-time strategy games require decisions within strict time limits (milliseconds/seconds), clashing with the massive processing time required by unoptimized tree searches.
3. Memory Limits: Storing large game subtrees exhausts RAM and recursion stack limits.

---

iii. How Alpha-Beta Pruning Improves Efficiency

- Alpha and Beta Thresholds: Tracks 'alpha' (best score Maximizer can guarantee) and 'beta' (best score Minimizer can guarantee).

- Pruning Cutoffs: If a branch yields a worse outcome than what a player can already secure elsewhere, the remaining sub-branch is pruned immediately because the rational opponent will never allow it.
- Efficiency Gain: Reduces time complexity from O(b^m) to O(b^(m/2)) in the best case, effectively doubling the search depth achievable within the same time limit.

---

iv. Design of an Evaluation Function and Justification of Factors

Formula: eval(s) = w1*(Material) + w2*(Territory/Resources) + w3*(Mobility) + w4*(Tactical Positioning)

Justified Factors:

1. Material Advantage (w1): Relative count and strength of active units/structures.
2. Territory & Resource Control (w2): Control over strategic map points and economic generators.
3. Mobility (w3): Number of valid movement and attack options, ensuring tactical flexibility.
4. Tactical Positioning (w4): Rewards high ground, chokepoints, and defensive formations.

---

v. Recommended Strategy for Balancing Optimality and Real-Time Performance

1. Depth-Limited Search: Truncate search at depth 'd' and evaluate leaf states using the heuristic evaluation function.
2. Iterative Deepening: Incrementally search deeper (depth 1, 2, 3...) until time expires, ensuring a valid move is always ready.
3. Move Ordering: Prioritize testing historically strong moves first to maximize Alpha-Beta pruning cutoffs.
4. Transposition Tables: Cache evaluated game states to avoid redundant calculations when identical board configurations are reached via different move sequences.